BSc Hons in Computing

2024.2

Problem Solving with Data Structures and Algorithms (PDSA-II)

Group Report: Minimum Cost Assignment Problem

Coursework 1

Leader Name: E M A S B EKANAYAKA
Cov. Index: 16110762
NIBM Index: COBSCCOMP242P-065

Member Name: M D S Y RANARAJA
Cov. Index: 16116133
NIBM Index: COBSCCOMP242P-075

Faculty of Engineering, Environment and Computing
School of Computing, Electronics and Mathematics
Coventry University

School of Computing and Engineering
National Institute of Business Management

List of Figures

1 Code segment where `N` is generated randomly between 50 and 100. . . . 1
2 Code segment where each cost matrix value is generated randomly from $20 to $200. 1
3 Code segment measuring and recording execution time for each algorithm. 2
4 Core implementation of the Greedy assignment algorithm. 2
5 Core implementation of the Hungarian algorithm. 3
6 Code segment saving `player_name` and related round data to the database. 3
7 Backend database insertion logic used during round result persistence. . 4
8 Normalized SQLite schema used for game rounds and algorithm results. 4
9 Frontend prompt where the user can enter their name before solving a round. 5
10 Unit tests validating greedy assignment behavior. 6
11 Unit tests validating Hungarian optimality and constraints. 7

List of Tables

1 Group Contribution and Marks iv

Contents

Contribution Table **iv**

1 Minimum Cost Problem **1**

1.1 Functionality 1

1.1.1 Code Segment: N is changing randomly from 50 to 100 in each game round 1

1.1.2 Code Segment: Cost of assigning each task to each employee changing randomly from $20 to $200 1

1.1.3 Code Segment: Record in the database how long it takes to find the minimum cost using each algorithm 1

1.1.4 Code Segment: Two different appropriate algorithms 2

1.1.5 Code Segment: Save that person's name along with the correct response in the database 3

1.1.6 Normalized DB Table Structure used for this Game Option . . . 4

1.2 UI 5

1.2.1 UI screenshot when asking user to enter the inputs & answers . 5

1.2.2 Validations and Exception Handling 5

1.3 Testing 6

1.3.1 Code Segment: Unit Testing 6

Contribution Table

Table 1: Group Contribution and Marks

Index	**NIBM Index**	**Marks /10**	**Contribution**
16110762	COBSCCOMP242P-065	10	Backend – algorithms, database, API, unit tests
16116133	COBSCCOMP242P-075	10	Frontend – React UI, game screens, timing chart

1. Minimum Cost Problem

1.1. Functionality

1.1.1. Code Segment: N is changing randomly from 50 to 100 in each game round

In the FastAPI backend, the number of employees/tasks N is selected randomly at the strt of each round between 50 and 100 using Python's `random.randint` function.

1. `N` changes randomly from 50 to 100 each game round

- File: main.py
- Lines: 34-42

```python
@app.get("/api/round/new", response_model=NewRoundResponse)
def new_round():
    """Generate a new game round: random N and a random cost matrix."""
    n = random.randint(50, 100)
    cost_matrix = [
        [random.randint(20, 200) for _ in range(n)]
        for _ in range(n)
    ]
    return NewRoundResponse(n=n, cost_matrix=cost_matrix)
```

Figure 1: Code segment where `N` is generated randomly between 50 and 100.

1.1.2. Code Segment: Cost of assigning each task to each employee changing randomly from $20 to $200

After determining N, the application generates an $N \times N$ matrix to represent the costs. Each cordinate c_{ij} is populated randomly via `random.randint(20, 200)`.

2. Cost matrix values change randomly from $20 to $200

- File: main.py
- Lines: 37-41

```python
n = random.randint(50, 100)
cost_matrix = [
    [random.randint(20, 200) for _ in range(n)]
    for _ in range(n)
]
```

Figure 2: Code segment where each cost matrix value is generated randomly from $20 to $200.

1.1.3. Code Segment: Record in the database how long it takes to find the minimum cost using each algorithm

The system executes both algorithms during the resllution phase and strictly measures the computing time using `time.perf_counter()`. The elapsed `time_ms` is then stored in the `algorithm_results` SQLite table.

4. Record time taken for each algorithm in the database

- File: main.py
- Lines: 77-79, 109-117, 120-124

```
            t0 = time.perf_counter()
            sub_assignment, _, sub_steps = algo_fn(sub_matrix)
            elapsed_ms = (time.perf_counter() - t0) * 1000
...
        results.append(
            AlgorithmResult(
                algorithm_name=algo_name,
                assignment=full_assignment,
                total_cost=total_cost,
                time_ms=round(elapsed_ms, 4),
                steps=steps,
            )
        )
...
    round_id = save_round(
        n,
        [{"algorithm_name": r.algorithm_name, "total_cost": r.total_cost, "time_ms": r.time_ms} for r in results],
        player_name=body.player_name or None,
        user_total_cost=user_total_cost,
    )
```

Figure 3: Code segment measuring and recording execution time for each algorithm.

1.1.4. Code Segment: Two different appropriate algorithms

The application evaluates answers using two assignment algorithm techniques:

1. **Greedy Approach**: A sub-optimal but fast $O(n^2)$ heuristic.
2. **Hungarian Algorithm (Kuhn-Munkres)**: An optimal $O(n^3)$ mathematical method that guarantes the absolute minimum total cost.

- Greedy algorithm implementation:
  - File: greedy.py
  - Lines: 14-67

```
def greedy_assignment(
    cost_matrix: list[list[int]],
) -> tuple[list[int], int, list[dict]]:
    ...
    for employee in range(n):
        best_task = -1
        best_cost = float("inf")
        candidates: list[tuple[int, int]] = []

        for task in range(n):
            if task not in assigned_tasks:
                candidates.append((task, cost_matrix[employee][task]))
                if cost_matrix[employee][task] < best_cost:
                    best_cost = cost_matrix[employee][task]
                    best_task = task

        assignment[employee] = best_task
        assigned_tasks.add(best_task)
        running_total += int(best_cost)
        ...
```

Figure 4: Core implementation of the Greedy assignment algorithm.

- Hungarian algorithm implementation:
  - File: hungarian.py
  - Lines: 19-60

```python
def hungarian_assignment(
    cost_matrix: list[list[int]],
) -> tuple[list[int], int, list[dict]]:
    ...
    # Step 1 — Row reduction
    for i in range(n):
        row_min = min(matrix[i])
        for j in range(n):
            matrix[i][j] -= row_min

    # Step 2 — Column reduction
    for j in range(n):
        col_min = min(matrix[i][j] for i in range(n))
        for i in range(n):
            matrix[i][j] -= col_min

    # Iterate until minimum line cover equals n
    while True:
        row_cover, col_cover = _min_line_cover(matrix, n)
        if len(row_cover) + len(col_cover) >= n:
            break
        _adjust_matrix(matrix, n, row_cover, col_cover)
```

Figure 5: Core implementation of the Hungarian algorithm.

1.1.5. Code Segment: Save that person's name along with the correct response in the database

Players can optionally input their name when playing the Mixed or Manual mode. The system captures their user assigned cost alongside their name and links it to the algorithm results.

5. Save player name to the database

- File: main.py
- Lines: 120-124, 140-149

```python
    round_id = save_round(
        n,
        [{"algorithm_name": r.algorithm_name, "total_cost": r.total_cost, "time_ms": r.time_ms} for r in results],
        player_name=body.player_name or None,
        user_total_cost=user_total_cost,
    )
```

```python
@app.post("/api/rounds/update-player-name")
def update_rounds_player_name(body: UpdatePlayerNameRequest):
    """Update player_name for a list of rounds (used when game ends)."""
    if not body.round_ids:
        raise HTTPException(status_code=400, detail="round_ids cannot be empty")
    if not body.player_name or not body.player_name.strip():
        raise HTTPException(status_code=400, detail="player_name cannot be empty")

    update_player_name(body.round_ids, body.player_name.strip())
    return {"status": "ok", "updated_rounds": len(body.round_ids)}
```

Figure 6: Code segment saving `player_name` and related round data to the database.

- Database save logic:
  - File: database.py
  - Lines: 43-58

```python
def save_round(n: int, results: list[dict], player_name: str | None = None, user_total_cost: int | None = None) -> int:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO game_rounds (n, player_name, user_total_cost) VALUES (?, ?, ?)",
        (n, player_name, user_total_cost),
    )
    round_id = cursor.lastrowid
    cursor.executemany(
        "INSERT INTO algorithm_results (round_id, algorithm_name, total_cost, time_ms) VALUES (?, ?, ?, ?)",
        [(round_id, r["algorithm_name"], r["total_cost"], r["time_ms"]) for r in results],
    )
    conn.commit()
    conn.close()
    return round_id
```

Figure 7: Backend database insertion logic used during round result persistence.

1.1.6. Normalized DB Table Structure used for this Game Option

Two tables are utilised to store and map the gaming records effectively in a normalized fashion:

- `game_rounds`: Tracks the overarching session details including N, the player's name, their user cost, and timestamp.
- `algorithm_results`: Stores a row mapping per algorithm tested linked via a `round_id` foreign key, containing properties like algorithm exactness (total cost) and efficiency (time taken).

- Database schema and normalized structure:
  - File: database.py
  - Lines: 16-31

```sql
CREATE TABLE IF NOT EXISTS game_rounds (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    n               INTEGER NOT NULL,
    player_name     TEXT,
    user_total_cost INTEGER,
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS algorithm_results (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    round_id       INTEGER NOT NULL REFERENCES game_rounds(id),
    algorithm_name TEXT NOT NULL,
    total_cost     INTEGER NOT NULL,
    time_ms        REAL NOT NULL
);
```

Figure 8: Normalized SQLite schema used for game rounds and algorithm results.

1.2. UI

1.2.1. UI screenshot when asking user to enter the inputs & answers

The frontend built using React and Vite features intuitive prompts allowing users to decide if they wish to compute automaticaly, input answers manually through the matrix, and define their usernames.

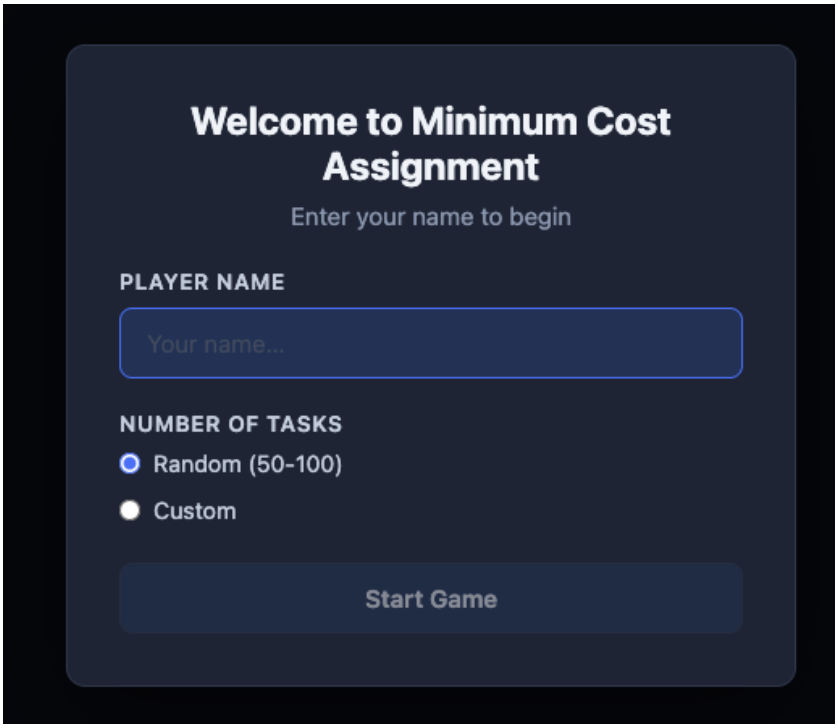

Figure 9: Frontend prompt where the user can enter their name before solving a round.

1.2.2. Validations and Exception Handling

Robust validation occurs both at the client and server levels:

- **Frontend (React)**: Game input relies on deterministic UI flows preventing illogical inputs (e.g. assigning one employee to multiple tasks inadvertently triggers UI reassignment toggles instead of crashing). Missing configuration paths are guarded using standard component state checks.
- **Backend (FastAPI)**: Pydantic models automatically validate incoming `POST/api/round/solve` requests. If incorrect data structures, mismatched N, or erroneous identifiers are pushed, FastAPI intercepts and returns 422 Unprocessable Entity responses. The database module utilises synchronous error catching avoiding data corruption from unique constraint or foreign key violations.

1.3. Testing

1.3.1. Code Segment: Unit Testing

We applied `pytest` for comprehensive automated validation. This contains isolated unit tests ensuring algorithm logic is strictly accurate. The testing checks for constraints such as verifying the Hungarian algorithm always finds a lower or equal total cost compared to the greedy heuristic.

```python
import pytest
from algorithms.greedy import greedy_assignment
from algorithms.hungarian import hungarian_assignment


# ---------------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------------

def is_valid_assignment(assignment: list[int], n: int) -> bool:
    """Every employee has a unique task in range [0, n)."""
    return sorted(assignment) == list(range(n))


def is_valid_steps(steps: list[dict], n: int) -> bool:
    """Steps list covers all n employees with required keys."""
    if len(steps) != n:
        return False
    required = {"employee", "task
                (variable) s: dict[Unknown, Unknown]
    return all(required.issubset(s.keys()) for s in steps)


# ---------------------------------------------------------------------------
# Greedy tests
# ---------------------------------------------------------------------------

class TestGreedyAssignment:

    def test_returns_valid_assignment_2x2(self):
        matrix = [[9, 2], [3, 8]]
        assignment, cost, steps = greedy_assignment(matrix)
        assert is_valid_assignment(assignment, 2)
        assert cost == sum(matrix[i][assignment[i]] for i in range(2))

    def test_returns_valid_assignment_3x3(self):
        matrix = [[4, 1, 3], [2, 0, 5], [3, 2, 2]]
        assignment, cost, steps = greedy_assignment(matrix)
        assert is_valid_assignment(assignment, 3)
        assert cost == sum(matrix[i][assignment[i]] for i in range(3))
```

Figure 10: Unit tests validating greedy assignment behavior.

```python
# ----------------------------------------------------------------------------
# Hungarian tests
# ----------------------------------------------------------------------------

class TestHungarianAssignment:

    def test_optimal_2x2(self):
        # Optimal: employee 0→task 1 (cost 2), employee 1→task 0 (cost 3) = 5
        matrix = [[9, 2], [3, 8]]
        assignment, cost, steps = hungarian_assignment(matrix)
        assert is_valid_assignment(assignment, 2)
        assert cost == 5

    def test_optimal_3x3_known_solution(self):
        # Optimal: 0→1 (1), 1→2 (1), 2→0 (1) = 3
        matrix = [[3, 1, 2], [2, 3, 1], [1, 2, 3]]
        assignment, cost, steps = hungarian_assignment(matrix)
        assert is_valid_assignment(assignment, 3)
        assert cost == 3

    def test_optimal_4x4_known(self):
        matrix = [
            [9,  2,  7,  8],
            [6,  4,  3,  7],
            [5,  8,  1,  8],
            [7,  6,  9,  4],
        ]
        # Brute-force verified optimal: 0→1 (2), 1→0 (6), 2→2 (1), 3→3 (4) = 13
        assignment, cost, steps = hungarian_assignment(matrix)
        assert is_valid_assignment(assignment, 4)
        assert cost == 13

    def test_returns_valid_assignment_single(self):
        matrix = [[99]]
        assignment, cost, steps = hungarian_assignment(matrix)
        assert assignment == [0]
        assert cost == 99

    def test_all_same_cost(self):
        matrix = [[7, 7, 7], [7, 7, 7], [7, 7, 7]]
        assignment, cost, steps = hungarian_assignment(matrix)
        assert is_valid_assignment(assignment, 3)
        assert cost == 21
```

Figure 11: Unit tests validating Hungarian optimality and constraints.